

Dokumentation der Pi-Platzierung auf der Karte

Dieses Kapitel beschreibt, wie die drei Kamerastationen (Pi 1–3) auf einer echten Satellitenkarte gesetzt werden und wie aus ihren Breiten- und Längengraden die lokalen ENU-Meter entstehen, mit denen die Simulation rechnet. Umgesetzt ist die Karte in

`helper_functions/pi_map.py`, die Koordinatenumrechnung in `helper_functions/geo_export.py`; eingebunden ist beides im Tab „Kameras“ von `webinterface_geo.py`.

Der Hintergrund: Die gesamte Simulation rechnet in ENU-Metern relativ zu Pi 1. Diese Meterwerte mussten bisher von Hand in die Tabelle eingetragen werden, ohne jeden Bezug zu einem realen Ort. Über die Karte wird stattdessen der tatsächliche Standort gewählt — die Meterwerte ergeben sich daraus automatisch, und umgekehrt bekommt die gesamte Simulation dadurch einen Ortsbezug, der später den Export und die Navigation zum geschätzten Revier möglich macht.

Hinweis zur realen Anwendung: Dieser Abschnitt ist nicht auf die Simulation beschränkt. Genau diesen Weg geht ein Livesystem beim Aufbau im Feld: Die Standorte der Pis werden per GPS oder auf der Karte bestimmt, und `wgs84_to_enu()` überführt sie in das Koordinatensystem, in dem Trajektorienberechnung und Vektor-Überschneidung arbeiten. Das entspricht Schritt 3 der Dokumentation „Überführung der Trajektorie Berechnung ins Live-System“ („Geografische Positionsdaten werden in ENU-Koordinaten umgerechnet“).

Datenfluss

```
pi-setup-table.data (Breite / Länge je Station)
|
▼
build_pi_map_figure() → Satellitenkarte mit Markern und ENU-Achsenlinien
|
| Nutzer verschiebt die Karte unter das Fadenkreuz
|
▼
pi-map.layoutData → center_from_layout() → pi-map-center (Store)
|
| Nutzer klickt „Pi N hierhin“
|
▼
place_pi_on_map() → schreibt Breite / Länge der Kartenmitte in die
Tabellenzeile
|
▼
derive_pi_positions() → wgs84_to_enu() je Station → Spalten X / Y in
Metern
```

|
▼
pi-setup-table.data → 3D-Ansicht, 2D-Kamerabild, Projektionen, GeoJSON-Export

Die Bedienung besteht aus drei Schritten:

1. **Karte positionieren** – Kartenausschnitt so verschieben, dass das Fadenkreuz auf dem gewünschten Standort liegt.
2. **Station setzen** – per Button die Breiten-/Längengrade der Kartenmitte in die gewählte Zeile übernehmen.
3. **Umrechnen** – die Koordinaten aller Stationen werden in ENU-Meter relativ zu Pi 1 überführt.

1. Kartenquelle

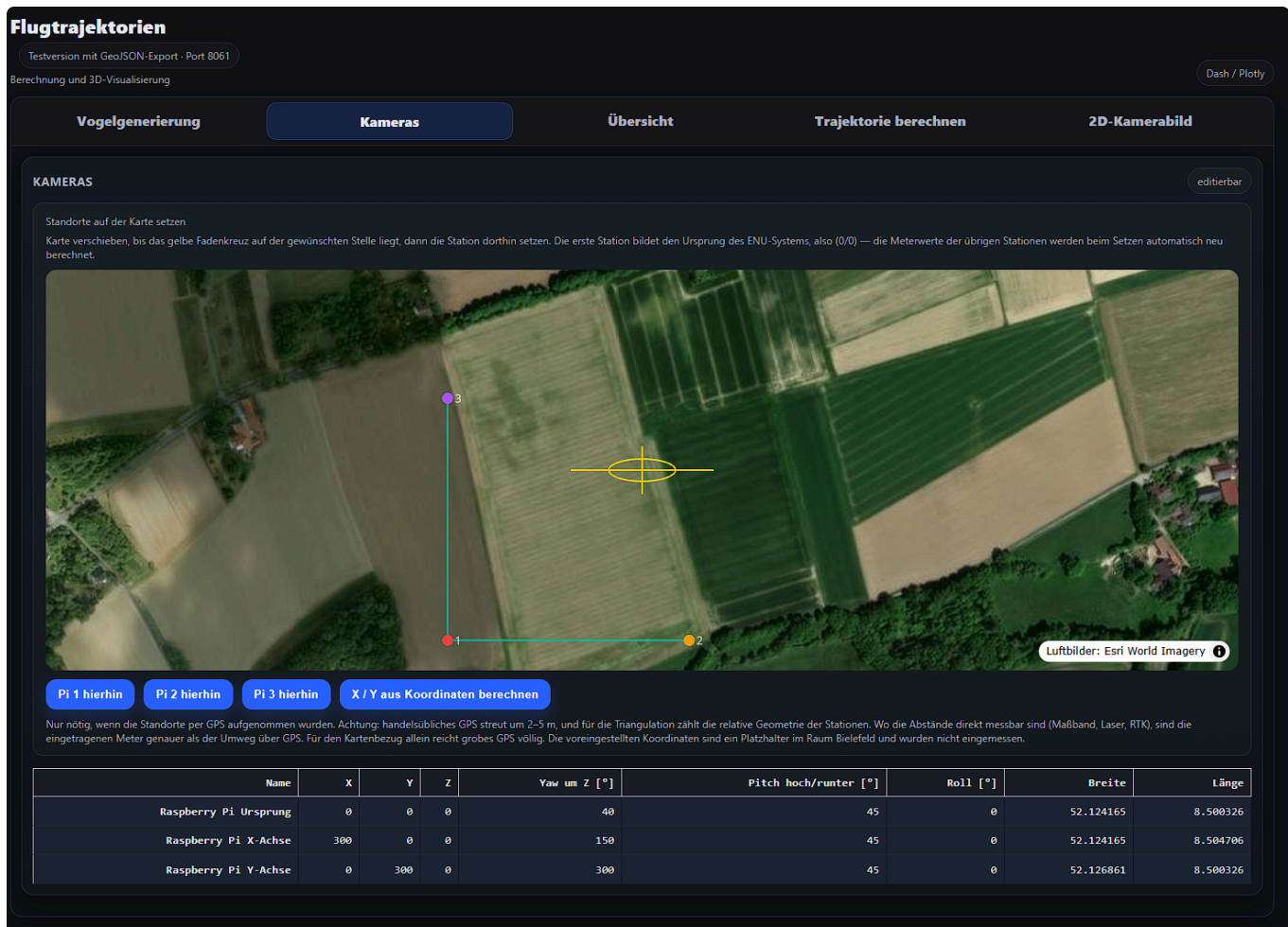
Verwendet werden die Luftbildkacheln von **Esri World Imagery**, eingebunden als Raster-Layer in eine `go.Scattermap`-Figur. Sie sind ohne API-Schlüssel abrufbar.

Google Maps und Google Earth lassen sich auf diesem Weg **nicht** einbinden: Deren JavaScript-API verlangt einen kostenpflichtigen Schlüssel, und das Einbetten der Kacheln außerhalb dieser API ist nicht zulässig. Inhaltlich ist die Darstellung gleichwertig — es sind Luftbilder derselben Flächen, nur von einem anderen Anbieter. Für die spätere Navigation zum geschätzten Revier wird dennoch direkt zu Google Maps verlinkt (siehe Doku „Revierschätzung und Flugprofile“), weil dort keine API eingebettet, sondern lediglich eine URL geöffnet wird — das ist lizenzrechtlich unproblematisch und funktioniert ohne Schlüssel.

2. Bedienung: Fadenkreuz statt Klick

Plotly meldet Klicks ausschließlich auf Datenpunkten, nicht auf freier Kartenfläche. Ein Klick „irgendwohin“ auf die Karte ist damit technisch nicht auswertbar. Stattdessen liegt ein festes gelbes Fadenkreuz in der Kartenmitte: Die Karte wird verschoben, bis die gewünschte Stelle darunter liegt, dann setzt ein Button die Station dorthin. Das entspricht der Bedienung, die aus gängigen Karten-Apps bei der Standortwahl bekannt ist.

Technisch wird das Fadenkreuz nicht als Datenpunkt gezeichnet, sondern über `add_shape()` im **Papierkoordinatensystem** (`xref="paper"`, `yref="paper"`, Mittelpunkt `0.5 / 0.5`). Dadurch bleibt es beim Verschieben und Zoomen der Karte exakt mittig, statt mit der Karte mitzuwandern.



Im Bild sind alle Bestandteile zu sehen: die drei gesetzten Stationen als nummerierte farbige Marker, die türkisen Verbindungslinien von Pi 1 zu Pi 2 und Pi 3 (sie zeigen die X- und Y-Achse des aufgespannten ENU-Systems), das gelbe Fadenkreuz in der Kartenmitte, die vier Buttons und darunter die Stationstabelle mit den zusätzlichen Spalten **Breite** und **Länge**.

Kartenzeichnung (`build_pi_map_figure`)

- Ist Pi 1 bekannt, werden türkise Verbindungslinien von Pi 1 zu jeder weiteren gesetzten Station gezogen. Sie machen die Geometrie sichtbar, die für die Triangulation entscheidend ist.
- Jede Station erscheint als nummerierter, farbiger Marker (`PI_COLORS`), beschriftet mit ihrem Namen und ihren Koordinaten im Hover.
- Ohne vorgegebenen Ausschnitt berechnet `fit_view()` Mittelpunkt und Zoom automatisch aus der Ausdehnung der gesetzten Stationen. Die Zoomstufe wird dabei so gewählt, dass die Ausdehnung etwa 60 % der Kartenbreite einnimmt, und auf den Bereich 10–18 begrenzt:

```
span = max(Nord-Ausdehnung, Ost-Ausdehnung, 50 m)
zoom = log2( 156543,03 · cos(lat) · 600 / (span · 1,7) )
```

- `uirevision="pi-map"` sorgt dafür, dass Plotly den gewählten Ausschnitt beim Neuzeichnen der Figur beibehält, statt auf die Ausgangsansicht zurückzuspringen.

3. Kartenmittelpunkt merken (`center_from_relayout` , `remember_map_center`)

Damit „Station auf die Kartenmitte setzen" funktioniert, muss bekannt sein, wo die Mitte gerade liegt. Jede Kartenbewegung löst ein `relayoutData` -Ereignis aus, das in den Store `pi-map-center` geschrieben wird.

Plotly liefert dabei je nach Aktion unterschiedliche Formen — mal `map.center` als verschachteltes Dict, mal die Einzelschlüssel `map.center.lat` / `map.center.lon` / `map.zoom`. `center_from_relayout()` behandelt beide Formen und übernimmt fehlende Werte aus dem vorherigen Zustand, sodass ein unvollständiges Ereignis den gemerkten Ausschnitt nicht zerstört.

4. Station setzen (`place_pi_on_map`)

Die drei Buttons „Pi 1/2/3 hierhin" teilen sich einen Callback. Welche Zeile gemeint ist, entscheidet `ctx.triggered_id`:

```
index_by_button = {
    "btn-place-pi-1": 0,
    "btn-place-pi-2": 1,
    "btn-place-pi-3": 2,
}
index = index_by_button.get(ctx.triggered_id)
```

Die aktuelle Kartenmitte wird auf sechs Nachkommastellen gerundet (etwa 0,1 m Auflösung) in die Spalten `lat` / `lon` der gewählten Zeile geschrieben. Direkt im Anschluss ruft der Callback `derive_pi_positions()` auf, sodass die Spalten X / Y **aller** Stationen sofort neu berechnet werden — Koordinaten und Meterwerte der Tabelle können damit nie auseinanderlaufen. Die Statuszeile meldet anschließend, welche Station gesetzt wurde und welche Meterwerte sich daraus ergeben haben.

5. Georeferenzierung: ENU ↔ WGS84

ENU (East-North-Up) ist ein lokales Tangentialsystem: Meter relativ zu einem Ursprung. Ohne die Angabe, wo dieser Ursprung auf der Erde liegt, haben die Koordinaten keinen Ortsbezug. Dieser Referenzpunkt ist **nicht** als separates Eingabefeld umgesetzt, sondern direkt an die erste Zeile der Stationstabelle gekoppelt: Pi 1 steht im Standardaufbau bei ENU (0, 0, 0), also ist seine Breite/Länge zwangsläufig auch der Georeferenzpunkt der gesamten Simulation (`reference_from_pi_rows`).

Krümmungsradien des WGS84-Ellipsoids (`_curvature_radii`)

Verwendet wird eine lokale Tangentialebenen-Näherung erster Ordnung um den Referenzpunkt. Grundlage sind die beiden Krümmungsradien des Ellipsoids auf der Breite `lat`:

```
e2      = f · (2 - f)                                Exzentrizität2
N(lat) = a / √(1 - e2 · sin2(lat))                    Querkrümmungsradius
M(lat) = a · (1 - e2) / (1 - e2 · sin2(lat))1,5      Meridiankrümmungsradius
```

mit `a = 6.378.137,0 m` (große Halbachse) und `f = 1 / 298,257223563` (Abplattung).

`N` beschreibt, wie viel Meter ein Längengrad-Schritt in Ost-Richtung ausmacht, `M` dasselbe für einen Breitengrad-Schritt in Nord-Richtung. Beide sind breitenabhängig, weil die Erde keine Kugel ist.

Beide Umrechnungsrichtungen

```
enu_to_wgs84(east, north, lat0, lon0):
    lat = lat0 + grad( north / M )
    lon = lon0 + grad( east / (N · cos(lat0)) )

wgs84_to_enu(lat, lon, lat0, lon0):
    north = rad(lat - lat0) · M
    east  = rad(lon - lon0) · N · cos(lat0)
```

Der Faktor `cos(lat0)` in der Ost-Richtung berücksichtigt, dass die Längengrade zu den Polen hin zusammenlaufen: Ein Längengrad entspricht am Äquator rund 111 km, auf der Breite von Bielefeld nur noch etwa 68 km.

Die beiden Funktionen sind zueinander invers und lassen sich damit gegeneinander testen (`round_trip_error`): Eine ENU-Position nach WGS84 und wieder zurückgerechnet weicht bei den Ausmaßen dieser Simulation um weniger als 0,0001 mm vom Ausgangswert ab.

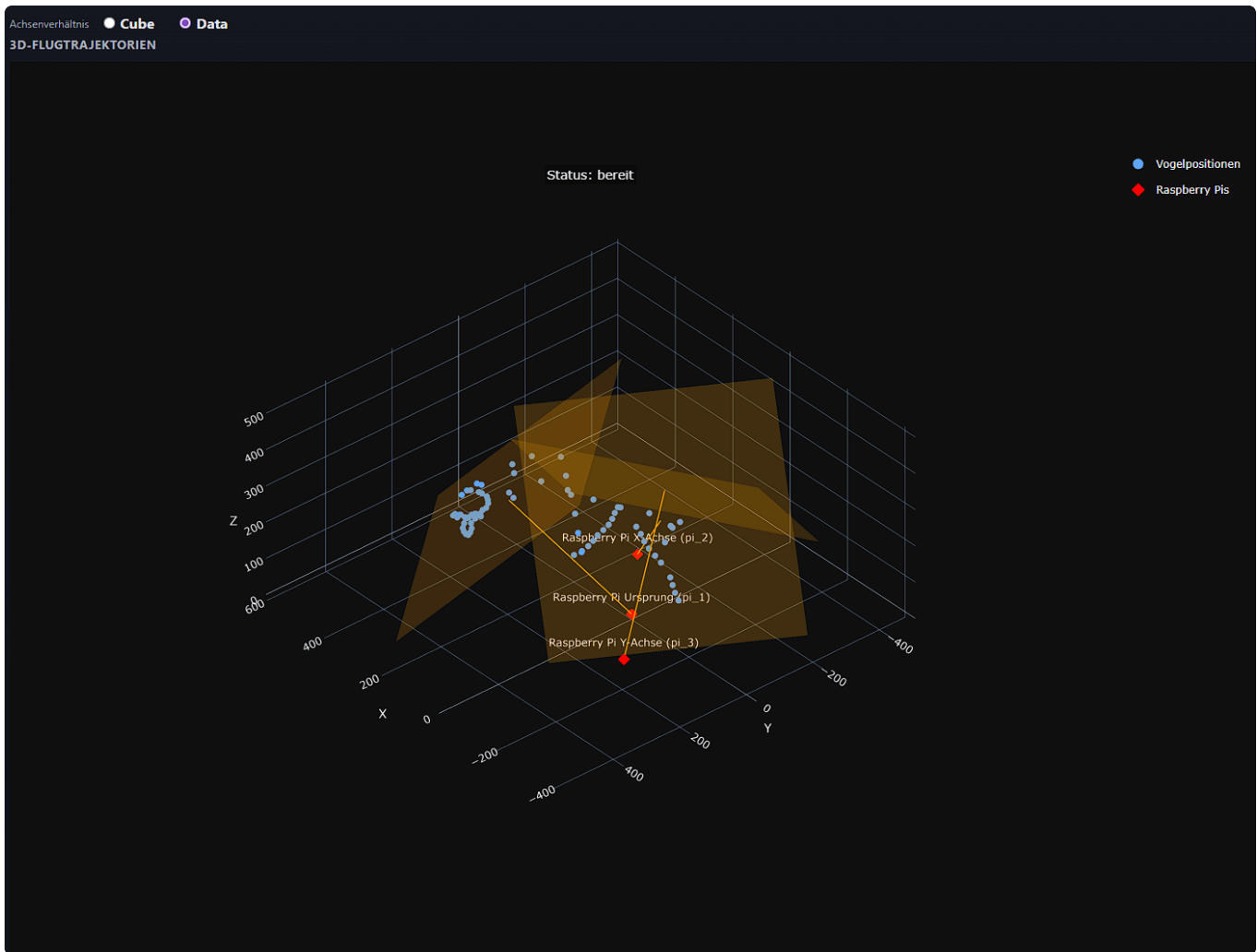
Genauigkeit der Näherung: Für die Ausdehnung dieser Simulation (einige hundert Meter) liegt der Fehler der Tangentialebenen-Näherung im Millimeterbereich, bis zu wenigen Kilometern im Zentimeterbereich. Für deutlich größere Gebiete wäre eine vollständige ECEF-Transformation nötig — für ein Kamerafeld mit wenigen hundert Metern Kantenlänge ist sie überflüssig.

6. Von Koordinaten zu Metern (`derive_pi_positions`)

Der Button „X / Y aus Koordinaten berechnen“ (bzw. jedes Setzen einer Station auf der Karte) rechnet die Tabelle durch:

```
Zeile 0 (Pi 1):    x = 0,00    y = 0,00                                → definiert den
Ursprung
Zeile i > 0:      (x, y) = wgs84_to_enu(lat_i, lon_i, lat_0, lon_0)
```

Die Werte werden auf zwei Nachkommastellen (Zentimeter) gerundet in die Spalten X und Y geschrieben. Die Statuszeile gibt zusätzlich die resultierenden Abstände zur ersten Station aus, damit sie sich mit tatsächlich gemessenen Abständen vergleichen lassen.



Im 3D-Modell erscheinen die Stationen anschließend an den umgerechneten Meterpositionen (rote Rauten), jeweils mit ihrer Blickrichtung und der daraus aufgespannten Kameraebene. Wird auf der Karte eine Station verschoben, ändern sich hier Position **und** Sichtfeld entsprechend — die Geometrie im Bild ist damit direkt das Ergebnis der Kartenplatzierung.

Genauigkeitshinweis (steht so auch als Hinweistext im Webinterface):

Handelsübliches GPS streut um 2–5 m. Für die Triangulation zählt die **relative** Geometrie der Stationen zueinander, dort wirkt sich dieser Fehler voll aus. Wo die Abstände direkt

gemessen werden können (Maßband, Laserentfernungsmesser, RTK-GPS), sind von Hand eingetragene Meter genauer als der Umweg über GPS oder Karte — die Spalten X und Y bleiben deshalb frei editierbar und überschreiben die berechneten Werte. Für den reinen Ortsbezug (Export, Navigation zum Revier) reicht grobes GPS dagegen völlig, weil dort nur die absolute Lage zählt, nicht die Genauigkeit im Zentimeterbereich.

Hinweis zu den Voreinstellungen: Die vorbelegten Koordinaten (`GEO_REFERENCE_LAT` / `GEO_REFERENCE_LON`) sind ein Platzhalter auf einer Ackerfläche im Raum Bielefeld und wurden **nicht** eingemessen. Sie dienen nur dazu, dass Karte, Export und Navigation ohne weitere Eingabe testbar sind.

7. Aktualisierung der abhängigen Ansichten

Die Pi-Positionen gehen in mehrere Darstellungen ein, die unterschiedlich schnell nachziehen können:

Ansicht	Verhalten bei geänderten Standorten
3D-Trajektorien	Wird sofort neu gezeichnet
2D-Kamerabild	Zeigt eine Warnung, dass die Projektionen veraltet sind
GeoJSON-Export	Verwendet beim nächsten Export automatisch den neuen Referenzpunkt

Beim 3D-Plot war dafür eine Korrektur nötig: Die Pi-Tabelle war im Callback als `State` eingebunden und wurde deshalb zwar gelesen, löste aber kein Neuzeichnen aus — der Plot zeigte die Kameras so lange an der alten Stelle, bis zufällig ein anderer Button gedrückt wurde. Der Wechsel auf `Input` behebt das:

```
# Als Input statt State: sonst behält der Plot die alten Kamerapositionen,  
# bis zufällig ein anderer Button gedrückt wird.  
Input("pi-setup-table", "data"),
```

Für das 2D-Kamerabild ist eine Live-Aktualisierung nicht sinnvoll, weil die projizierten Bildpunkte in `vogel_bilder.json` liegen und nur durch einen erneuten Durchlauf der Projektionsberechnung entstehen — das bei jeder Kartenbewegung auszulösen, würde die Bedienung ausbremsen. Stattdessen wird der Zustand **erkannt**:

`projection_matches_pi_setup()` vergleicht die in der JSON gespeicherten `pi_position` und `view_direction_vector` mit den aktuellen Tabellenwerten (Toleranz 0,01). Weichen sie ab, erscheint über dem Bild ein Hinweis und die Statuszeile meldet „Projektionen veraltet — Kamerastandorte wurden geändert“, statt stillschweigend ein falsches Bild zu zeigen.

Parameter

Parameter	Standardwert	Ort	Beschreibung
ESRI_WORLD_IMAGERY	Tile-URL-Vorlage	pi_map.py:30	Quelle der Luftbilder; kein API-Schlüssel
DEFAULT_ZOOM	15.5	pi_map.py:37	Zoomstufe ohne bzw. nur einer gesetzten Station
PI_COLORS	3 Farbwerte	pi_map.py:39	Markerfarben der Stationen
WGS84_SEMI_MAJOR_AXIS	6378137.0 m	geo_export.py:46	Große Halbachse des WGS84-Ellipsoids
WGS84_FLATTENING	1/298.257223563	geo_export.py:47	Abplattung des WGS84-Ellipsoids
WGS84_ECCENTRICITY_SQUARED	$f \cdot (2 - f)$	geo_export.py:48	Exzentrizität aus der Abplattung abgeleitet
GEO_REFERENCE_LAT / _LON	52.124165 / 8.500326	webinterface_geo.py:91	Voreingestellter Standort 1 (Platzhalter, nicht eingetragener Ort)

Zentrale Funktionen

Funktion	Datei	Aufgabe
build_pi_map_figure	pi_map.py	Baut die Satellitenkarte mit Markern, Achsenlinien und Fadenkreuz
pi_coordinates	pi_map.py	Liest Breite/Länge je Zeile der Stationstabelle
fit_view	pi_map.py	Kartenmittelpunkt und Zoom passend zu den gesetzten Stationen

Funktion	Datei	Aufgabe
center_from_layout	pi_map.py	Liest Mittelpunkt und Zoom aus Plotlys layoutData
_curvature_radii	geo_export.py	Quer- und Meridiankrümmungsradiu auf einer geografischen Breite
enu_to_wgs84	geo_export.py	ENU-Meter → Breite/Länge
wgs84_to_enu	geo_export.py	Breite/Länge → ENU-Meter (Richtung fürs Livesystem)
derive_pi_positions	geo_export.py	Rechnet die Koordinaten aller Stationen in ENU-Meter um
reference_from_pi_rows	geo_export.py	Liest den Georeferenzpunkt aus de ersten Stationszeile
round_trip_error	geo_export.py	Testhilfe: Hin- und Rückrechnung, verbleibender Fehler in Metern
remember_map_center	webinterface_geo.py	Callback: hält den Kartenausschnitt im Store fest
place_pi_on_map	webinterface_geo.py	Callback: setzt eine Station auf die Kartenmitt
update_pi_map	webinterface_geo.py	Callback: zeichnet die Karte bei geänderten Standorten neu
derive_positions_from_coordinates	webinterface_geo.py	Callback: Button „X / Y aus Koordinaten berechnen“
projection_matches_pi_setup	webinterface_geo.py	Prüft, ob die gespeicherten 2D-Projektionen noch zum Aufbau passen